

Deployment-guide

Sådan gør du Landmålerberegneren tilgængelig på internettet — trin for trin, til at gentage

Dette dokument er en praktisk checkliste over den fremgangsmåde, vi brugte til at gøre appen tilgængelig for andre på internettet — hosting af det visuelle på eget webhotel (One.com), kombineret med gratis serverless-funktioner (Vercel) til at skjule API-nøglen. Følg trinene i rækkefølge, hvis du skal gøre det samme igen på et andet projekt.

Hvorfor er det ikke bare "upload og færdig"?

Under udvikling bruger appen en lokal proxy (indbygget i `vite.config.js`), der kører sammen med `npm run dev` på din egen maskine. Den proxy skjuler din API-nøgle og videregiver kald til Dataforsyningen. Problemet: denne proxy er kun en del af *udviklingsserveren* — den følger ikke med, når du bygger appen til en almindelig, statisk hjemmeside med `npm run build`.

■■ Almindelig delt webhotel (som One.com) kan vise statiske filer (HTML/CSS/JS), men kan ikke køre en vedvarende Node.js-baggrundsproces. Derfor skal "nøgle-skjuler"-funktionen bo et andet sted.

Løsningen har to dele:

- **Det visuelle** (selve appen) → statiske filer, uploadet til dit eget webhotel.
- **API-nøgle-håndteringen** → to små "serverless functions", hostet gratis på Vercel, som kun har den ene opgave: modtage kald, tilføje nøglen usynligt, og sende videre til Dataforsyningen.

Overblik over filerne der skal bruges

Fil	Formål
<code>api/skelpunkter.js</code>	Serverless function der erstatter GraphQL-proxyen
<code>api/ortofoto.js</code>	Serverless function der erstatter WMTS-flise-proxyen
<code>src/utils/skelApi.js</code>	Opdateret til at kalde en konfigurerbar adresse (lokal proxy ELLER Vercel)
<code>src/components/MapView.jsx</code>	Samme — ortofoto-laget bruger også den konfigurerbare adresse
<code>vite.config.js</code>	Får tilføjet base-sti, hvis appen skal ligge i en undermappe
<code>.env.production</code>	Ny fil — fortæller build-processen, hvor Vercel-funktionerne bor

Trin for trin

1. Forbered en kopi af projektet

Behold din originale udviklingsmappe urørt. Kopiér den til en ny mappe (fx `mit-projekt-public`), og arbejd videre i kopien.

2. Opret serverless functions i en `api`-mappe

Opret en mappe kaldet `api` i projektets rod (samme niveau som `src`). Heri lægges én fil pr. proxy-rute fra din `vite.config.js`. Hver fil eksporterer en handler-funktion, der:

- Sætter CORS-headers, så browseren må kalde funktionen fra en anden adresse.
- Læser nøglen fra en miljøvariabel: `process.env.DIN_NOEGLE_NAVN`.
- Sender kaldet videre til den rigtige, eksterne tjeneste med nøglen tilføjet.
- Returnerer svaret tilbage til browseren.

3. Gør API-kaldene i appen konfigurerbare

Skift fra faste stier som `/api/...` til en variabel base-adresse, så koden virker begge steder:

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || ''
fetch(`${API_BASE_URL}/api/skelpunkter`, ...)
```

Uden en sat `VITE_API_BASE_URL` (lokal udvikling) peger kaldet på `'/api/...'`, som Vites egen proxy håndterer. Med den sat (produktionsbuild) peger kaldet direkte på Vercel.

4. Opret Vercel-konto og log ind fra terminalen

```
npm install -g vercel
vercel login
```

5. Deploy `api`-funktionerne til Vercel

Kør fra roden af din projekt-kopi:

```
vercel
```

Svar på spørgsmålene: opret nyt projekt, accepter standardnavn/mappe, lad Vercel selv opdage Vite. Du får en fast adresse tilbage, fx:

```
https://dit-projekt.vercel.app
```

6. Tilføj din API-nøgle til Vercel

```
vercel env add DIN_NOEGLE_NAVN production
```

Vælg "Sensitive" (Y), og indsæt kun selve nøgleværdien — ikke variabelnavnet. Deploy derefter igen, så ændringen tages i brug:

```
vercel --prod
```

7. Opret `.env.production` lokalt

I roden af projekt-kopien, ny fil:

```
VITE_API_BASE_URL=https://dit-projekt.vercel.app
```

8. (Hvis appen skal ligge i en undermappe) Sæt base-sti

Hvis appen skal ligge på `dindomæne.dk/undermappe/` i stedet for på selve forsiden, tilføjes dette i `vite.config.js`:

```
export default defineConfig({  
  base: '/undermappe/',  
  ...  
})
```

■■ Stien skal matche undermappens navn præcist, med skråstreg både foran og bagved.

9. Byg og test lokalt

```
npm run build  
npm run preview
```

Tjek at alt virker på den lokale preview-adresse, før du uploader — det er den hurtigste måde at fange fejl på.

10. Upload til webhotellet

Upload **indholdet af dist-mappen** (ikke selve mappen) til den ønskede placering på dit webhotel, via filhåndtering eller FTP. `api`-mappen skal **ikke** uploades hertil — den bor kun på Vercel.

Fejlfinding — det vi selv stødte på

Symptom	Løsning
Blank/hvid side efter upload	Tjek om base-stien i vite.config.js matcher undermappens navn præcist
404 på .js/.css-filer i konsollen	Se kildekoden (Ctrl+U) — hvis stierne mangler undermappen, er build'et lavet uden h
rmdir /s /q virker ikke	Det er PowerShell-syntaks der driller — brug i stedet: Remove-Item -Recurse -Force
npm install -g vercel giver advarsler	Ofte ufarligt — bekræft med "vercel --version" om det faktisk virker, før du fejlsøger v
Ændringer virker ikke efter deployment	Miljøvariabler og config-ændringer kræver en ny "vercel --prod" for at blive taget i br

Godt at huske

- Din API-nøgle bruges af **alle** besøgende på siden — det er én fælles nøgle, ikke individuelle logins.
- Nøglen ligger kun i Vercels miljøvariabler — aldrig i de filer, du uploader til webhotellet.
- Når du ændrer koden: byg (npm run build) og upload dist-mappen igen — hver gang.
- Din originale udviklingsmappe er urørt og fin til fortsat udvikling — test nye ting der først.

Udarbejdet som deployment-checkliste for Landmålerberegner-projektet — genbrugelig til fremtidige projekter.